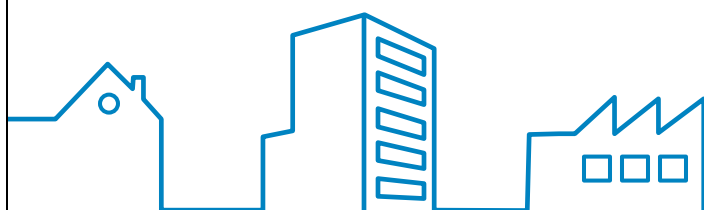




Progettazione e Realizzazione di un Veicolo Elettrico Teleoperato in CANOpen

*Tesi Sperimentale in collaborazione tra
Daikin Applied Europe e Università di Roma Tor Vergata*



- — 1. Introduzione
- — 2. Definizione del Problema
- — 3. Obiettivi di Progetto e Design
- — 4. Selezione Componenti
- — 5. Architettura di Comunicazione e Controllo
- — 6. Stato dei Lavori
- —
—
—



1. Introduzione

La realtà di DAE e l'importanza dell'automazione

La Realtà di Daikin Applied Europe - Cecchina

- Produzione di macchinari industriali per la climatizzazione
- Produzione Make-to-Order e poco standardizzata
- Forte crescita della domanda
 - Espansione della produzione e del personale
- Scarsa digitalizzazione dei processi produttivi (non R&D)
- Grande necessità di automatizzazione:
 - Gestione documenti
 - Tracciabilità dei componenti
 - **Logistica**



Fig. 1: Stabilimento di Cecchina (RM)



Fig. 2: Banco Prova Chiller

L'idea di un Veicolo Autonomo per il trasporto di sottoassiemi e semilavorati

Tipologia di movimentazione ad Oggi:

- Manuale
- O tramite muletti
- Basata su richieste giornaliere dei responsabili di produzione
- Ad alto costo per la mano d'opera
- Con notevoli ritardi e rallentamenti di produzione dovuti alla ricerca del pezzo

Tipologia di Movimentazione Futura:

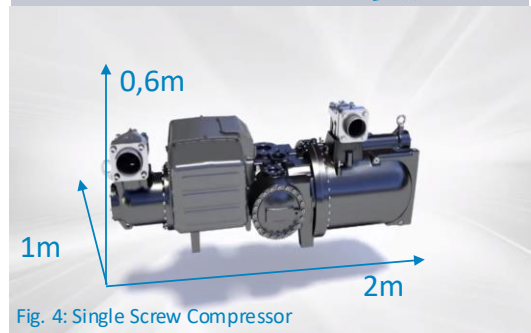
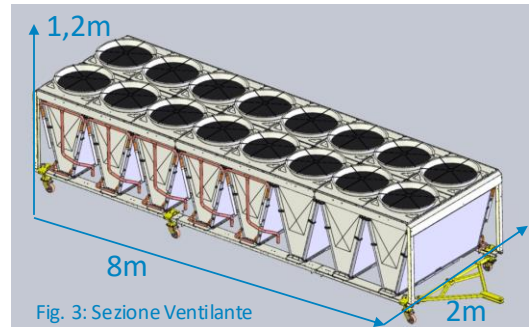
- Attuata con una flotta di veicoli elettrici
- Controllati tramite sistemi di guida autonoma / controllo remoto
- Mediante un sistema centralizzato di schedulazione e gestione della logistica
- Con un costo ammortizzabile negli anni
- Permettendo ottimizzazioni nei processi produttivi, dalla supply chain fino alla delivery

2. Definizione del Problema

Realizzare da zero un veicolo autonomo

Caratteristiche della Movimentazione desiderata

- Focus su una tipologia di produzione: Chiller
 - è composto da una *sezione ventilante* e dal sistema del circuito refrigerante: Scambiatore e **Compressore**
 - Una sez. ventilante (SV) è composta da *moduli*
 - Ogni modulo è circa 1x2m e il loro numero può variare da 4 a 12.
 - Un compressore, a seconda del modello, oscilla tra l'1.5 e le 2 tonnellate ma le dimensioni sono contenute, circa 1x2m



Caratteristiche della Movimentazione desiderata

- Focus su una tipologia di produzione:
Chiller
- Focus su un particolare sottoassieme:
Compressore
 - Supporti di appoggio standard con pedana
 - Dimensioni contenute e fisse
 - Carico elevato ma non eccessivo
 - Semplice da caricare sul veicolo

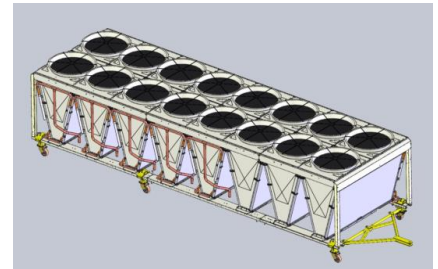


Fig. 3: Sezione Ventilante

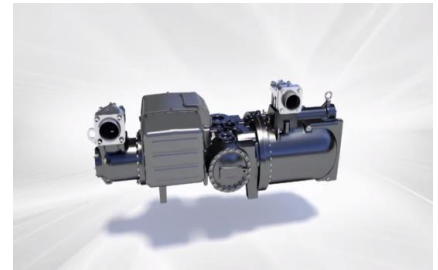


Fig. 4: Single Screw Compressor

Caratteristiche della Movimentazione desiderata

- Focus su una tipologia di produzione: Chiller
- Focus su un particolare sottoassieme: Compressore
- Trasporto dalla piazza di stoccaggio fino alla linea:
 1. Una volta prodotto viene stoccato all'aperto in una piazzola
 2. Deve essere caricato sul veicolo
 3. Trasportato verso il capannone Chiller, affrontando un percorso in esterna con dislivelli leggeri sotto i 10°
 4. Posizionato nella stazione di montaggio per essere scaricato dal veicolo
 5. Ritorno del veicolo alla stazione di attesa

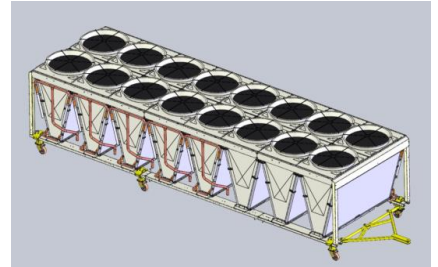


Fig. 3: Sezione Ventilante

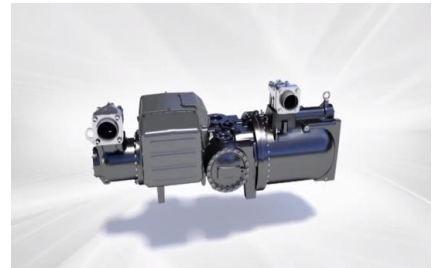
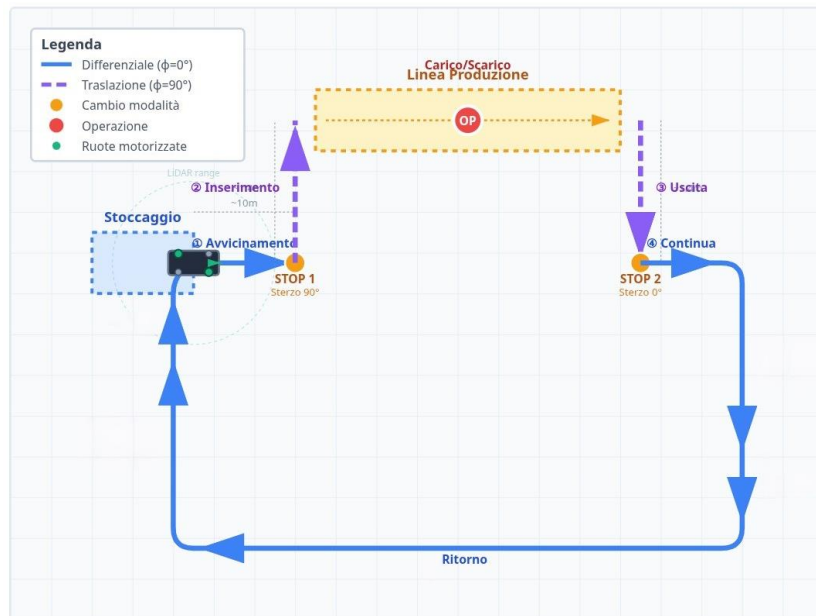


Fig. 4: Single Screw Compressor

Percorso Operativo previsto per il prototipo



10 Design della Soluzione e



1. Avvicinamento dalla piattaforma di stoccaggio alla stazione della linea di produzione
2. Inserimento nella stazione di produzione mediante traslazione rigida
3. Uscita dalla stazione di produzione mediante stazione rigida
4. Proseguimento e/o ritorno alla piattaforma di stoccaggio

Requisiti di alto livello

- Intelaiatura adatta al trasporto di carichi elevati
- Punti di ancoraggio per il payload
- Motori con coppia sufficiente a trasportare i carichi richiesti
- Meccanismi di sterzo
- Rilevazione degli ostacoli e localizzazione
- No telecamere (privacy)
- Batteria ad alta capacità per offrire servizio continuativo
- Velocità di crociera bassa (max 3 km/h)
- **Mantenere i costi più bassi possibili**

Obiettivi di progetto e

3. Design

Il prototipo, come farlo muovere e con cosa equipaggiarlo

Obiettivi di progetto

1. Selezione Hardware
2. Modellazione Matematica
3. Progettazione Meccanica
4. Progettazione Elettrica
5. Progettazione Architettura di Controllo
6. Progettazione Architettura di Comunicazione
7. Selezione dei Fornitori
8. Messa in opera (*in corso*)

Design del Prototipo – Selezione Componenti

- Telaio con profilati in acciaio
- Ruote motorizzate sterzanti
 - Solo 2 attuazioni, ognuna con motore di trazione e sterzo
 - 4 driver per motori elettrici
 - Encoder per ciascun motore
 - Gomme in poliuretano
- LiDAR + IMU
 - SLAM, feedback posa
- Batteria con chimica LiFePO4
 - Sicurezza, alta efficienza e capacità
- On-Board PC con CPU x86
 - Compatibilità con OS moderni
 - Flessibilità per sviluppi futuri

Design del Prototipo – Modellazione Matematica

- Biciclo con ruote attuate sulla diagonale del robot
- Gestione degli sterzi indipendente
- Gestione delle velocità tangenziali delle ruote indipendenti
- Vengono trascurati i disturbi causati dalle ruote passive
- Formulazione puramente cinematica
 - Con alcune criticità

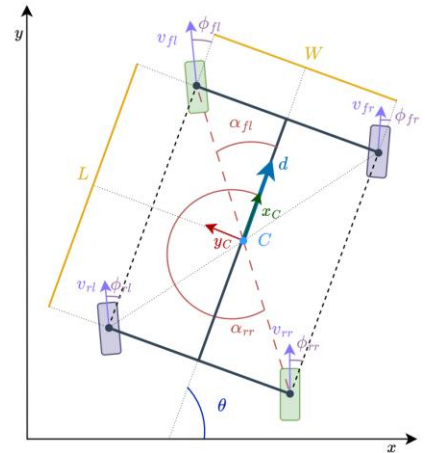


Fig. 7a: Schematizzazione del prototipo. In verde le ruote attuate

Design del Prototipo – Modellazione Matematica

- Posizione del centro di una ruota

$$\vec{p}_k = \begin{bmatrix} x \\ y \end{bmatrix} + \mathbf{R}(\theta) \vec{r}_k \quad \vec{r}_k = \rho \begin{bmatrix} \cos(\alpha) \\ \sin(\alpha) \end{bmatrix}$$

$$\rho = \frac{1}{2} \sqrt{L^2 + W^2}$$

- Velocità del centro di una ruota

$$\dot{\vec{p}}_k = \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} + \dot{\mathbf{R}}(\theta) \vec{r}_k + \mathbf{R}(\theta) \dot{\vec{r}}_k \quad \dot{\vec{r}}_k = 0$$

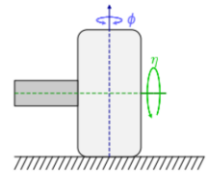
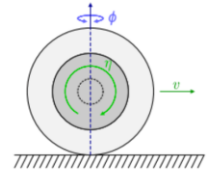


Fig. 7b: Cinematica di una ruota sterzante

Design del Prototipo – Modellazione Matematica

- Vincoli:

$$\begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = R\eta$$
$$\begin{bmatrix} \sin(\theta) & -\cos(\theta) & 0 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = 0$$

- Imponendo i vincoli

$$C(q)\dot{q} = b(u)$$



- Cinematica Diretta

$$\dot{q} = C(q)^\dagger b(u)$$

Design del Prototipo – Modellazione Matematica

- Stato con due ruote

$$q(t) = (x, y, \theta, \phi_f, \phi_r)$$

$$u = (\eta_f, \eta_r, \psi_f, \psi_r)$$

- Parametri: (r, ρ, α)

- Assunzioni: $\dot{\phi} = \psi$

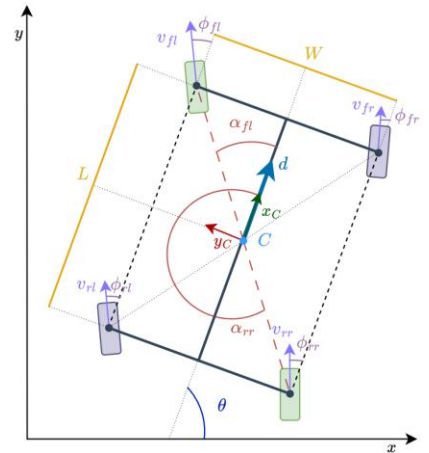


Fig. 7a: Schematizzazione del prototipo. In verde le ruote attuate

Design del Prototipo – Modellazione Matematica

- Rappresentazione nello spazio di stato

$$\dot{x} = \frac{r}{2} (\eta_f \cos(\phi_f + \theta) + \eta_r \cos(\phi_r + \theta))$$

$$\dot{y} = \frac{r}{2} (\eta_f \sin(\phi_f + \theta) + \eta_r \sin(\phi_r + \theta))$$

$$\dot{\theta} = \frac{r}{2\rho} (-\eta_f \sin(\alpha - \phi_f) + \eta_r \sin(\alpha - \phi_r))$$

$$\dot{\phi}_f = \psi_f$$

$$\dot{\phi}_r = \psi_r$$

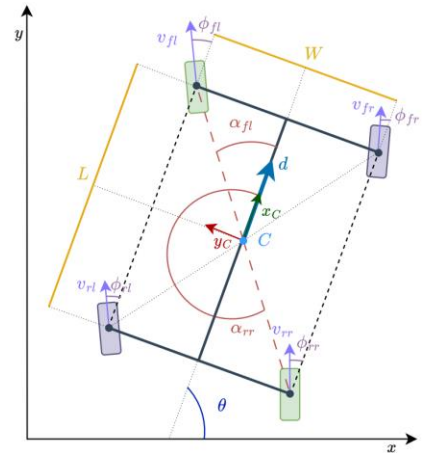


Fig. 7a: Schematizzazione del prototipo. In verde le ruote attuate

- Equivalentemente $\dot{q} = G(q)u$

- E' stata eseguita analisi della controllabilità costruendo l'algebra di Lie, impilando:

- E' stata verificata la controllabilità con rango 5



Design del Prototipo – Simulazione in Catena Aperta

Example 1: Straight Line Motion

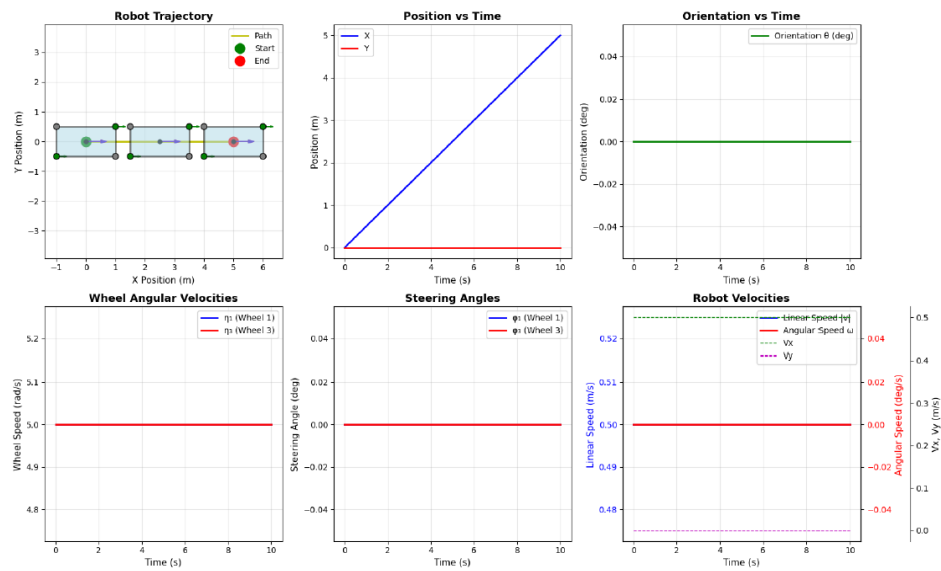


Fig. 7c: Moto Rettilineo



Design del Prototipo – Simulazione in Catena Aperta

Example 2: Circular Motion (Diff-Drive)

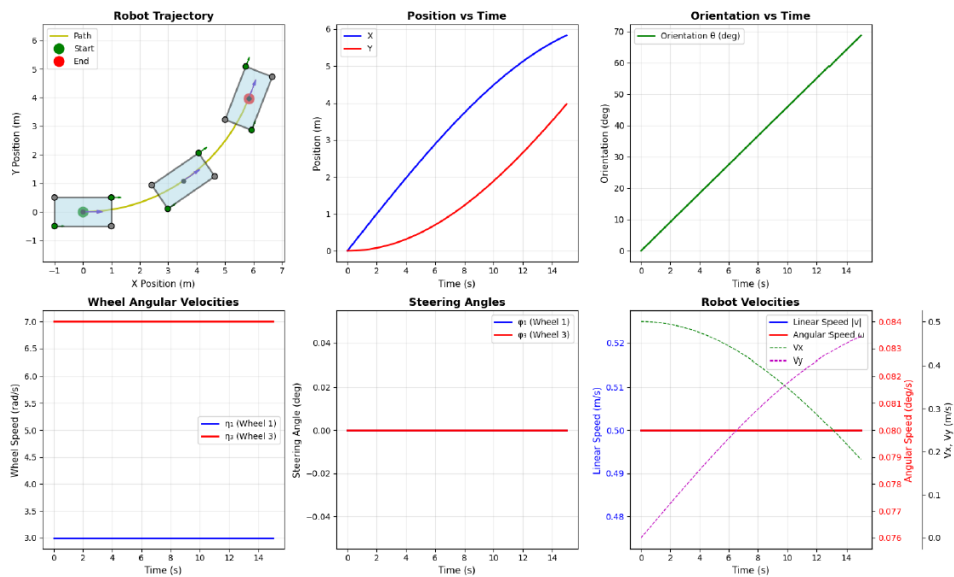


Fig. 7d: Moto Curvilineo



Design del Prototipo – Simulazione in Catena Aperta

Example 3: Differential Speed Control

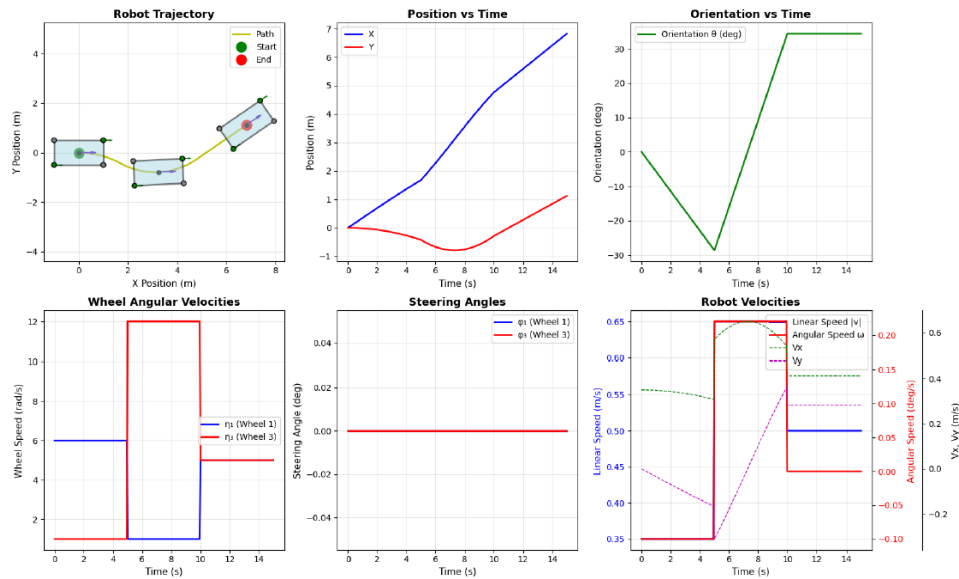


Fig. 7e: Moto misto



Design del Prototipo – Simulazione in Catena Aperta

Example 4: Square Pattern

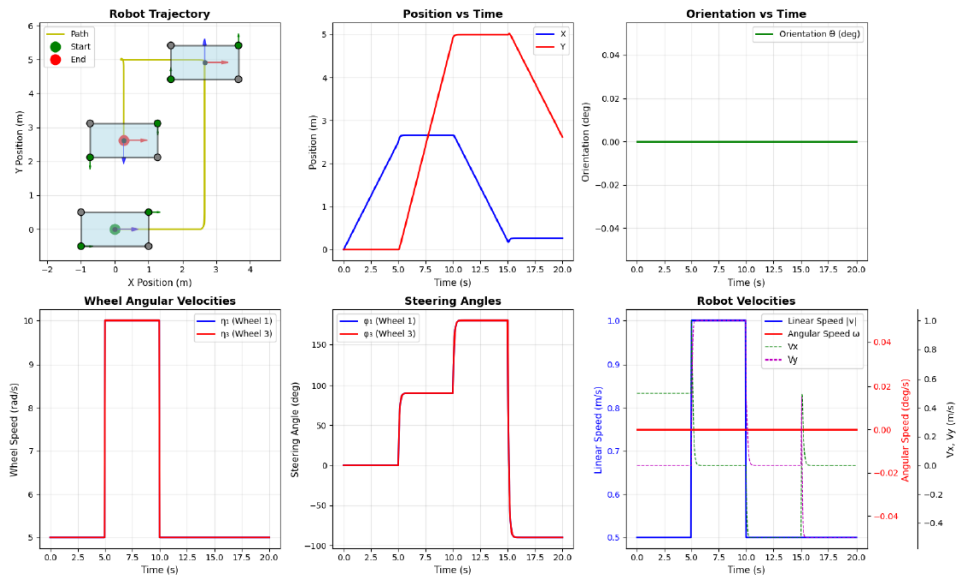


Fig. 7f: Moto su traiettoria rettangolare



Design del Prototipo – Simulazione in Catena Aperta

Example 5: In-Place Rotation

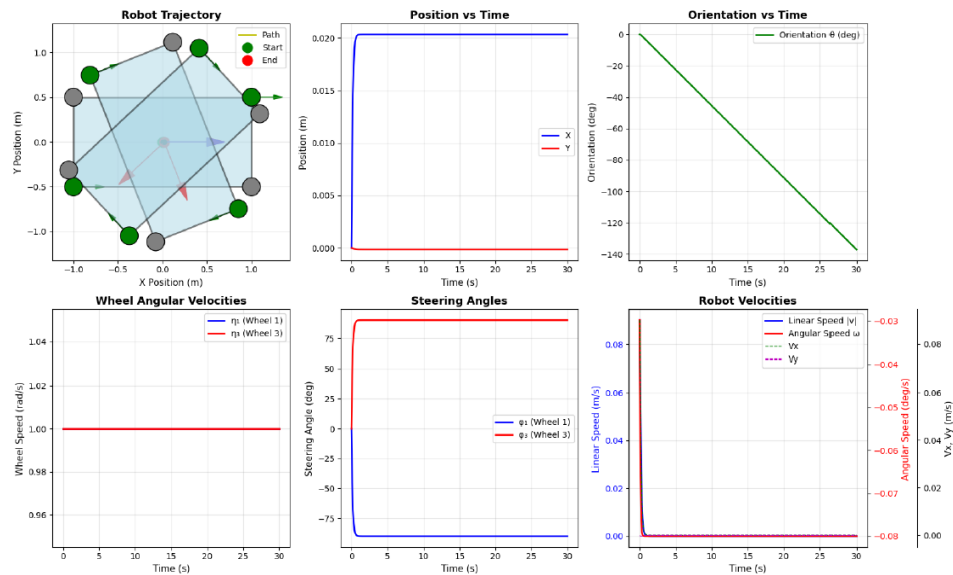


Fig. 7g: Rotazione sul posto



Design del Prototipo – Modellazione Matematica

- Assumendo di poter misurare tutto lo stato
 - Posa (x, y, θ) con algoritmo di SLAM
 - (ϕ_r, ϕ_f) dagli encoder agli sterzi
- Cinematica Inversa $u = G(q)^\dagger q$
- Controllo con PI per inseguire riferimenti di posizione e velocità per asservimento di traiettorie

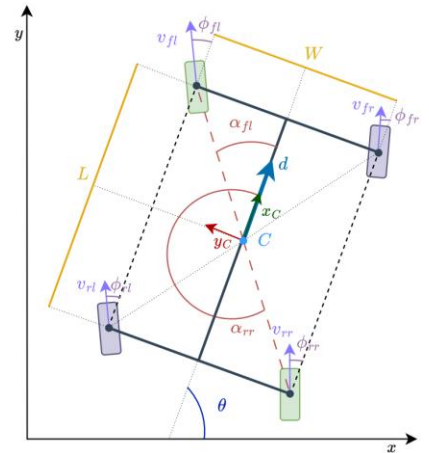


Fig. 7a: Schematizzazione del prototipo. In verde le ruote attuate

Design del Prototipo – Modellazione Matematica - Criticità

- I vincoli di *non-slittamento* non vengono sempre rispettati
- Non sono considerati limiti sulle variabili di controllo
- Non è tenuto conto di eventuali errori di misura
- Se la dimensione orizzontale del corpo è molto elevata rispetto quella verticale, il modello perde di senso
- Dinamiche degli sterzi semplificata
 - E' in corso lo sviluppo di un'approssimazione al primo ordine per considerare un ritardo di tempo
- Ci sono alcune configurazioni singolari non mappate (es: angoli uguali, velocità contrarie)
 - Risolvendole in cinematica inversa: problemi di ottimizzazione vincolata su velocità di rotazione delle ruote e valore degli sterzi
 - Risolvendole in cinematica diretta: modificare il modello cinematico inserendo sterzo di Ackermann o vincolando a priori le velocità tangenziali e gli orientamenti degli sterzi
 - Risolvendole in generazione di traiettorie con ottimizzazione di vincoli
- **Si assume che le condizioni operative non rientrino in queste casistiche**

Design del Prototipo – Progettazione Meccanica

- Progettazione virtuale in 3D con Solidworks
- Caratteristiche principali:
 - Dimensioni: 1x2x0.4 m
 - Realizzato con profilati in acciaio a sez. quadrata da 20mm, saldati
 - Peso complessivo di circa 500 kg
- Supporti extra:
 - Lamiere per alloggiamento componenti
 - Regolabili per l'alloggiamento del payload
- Progetto validato al sostegno dei carichi mediante analisi FEM
- Lavorazioni in acciaio realizzate da ditta fornitrice specializzata

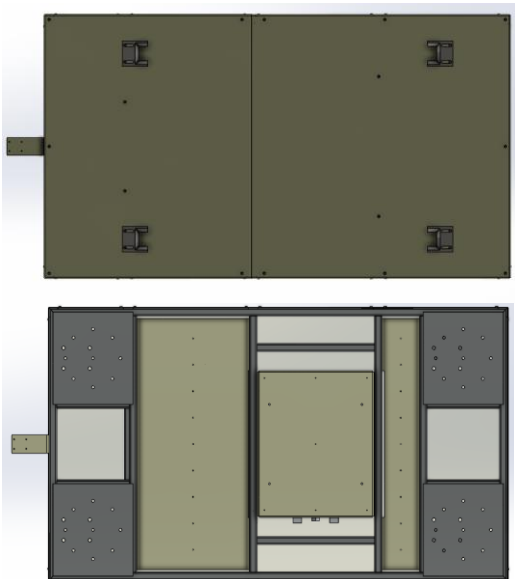


Fig. 5a 5b: Vista dall'alto e dal basso del telaio

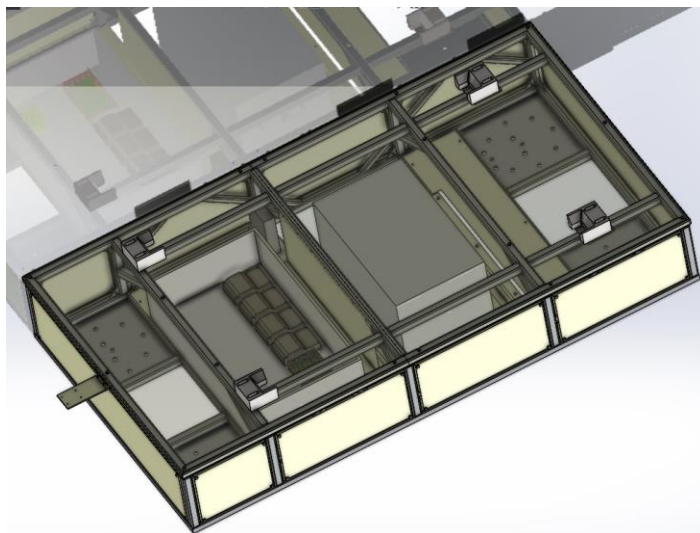


Fig. 6: Vista di taglio, senza i pannelli superiori. Si possono osservare la batteria e il quadro elettrico contenente i driver e il computer di bordo

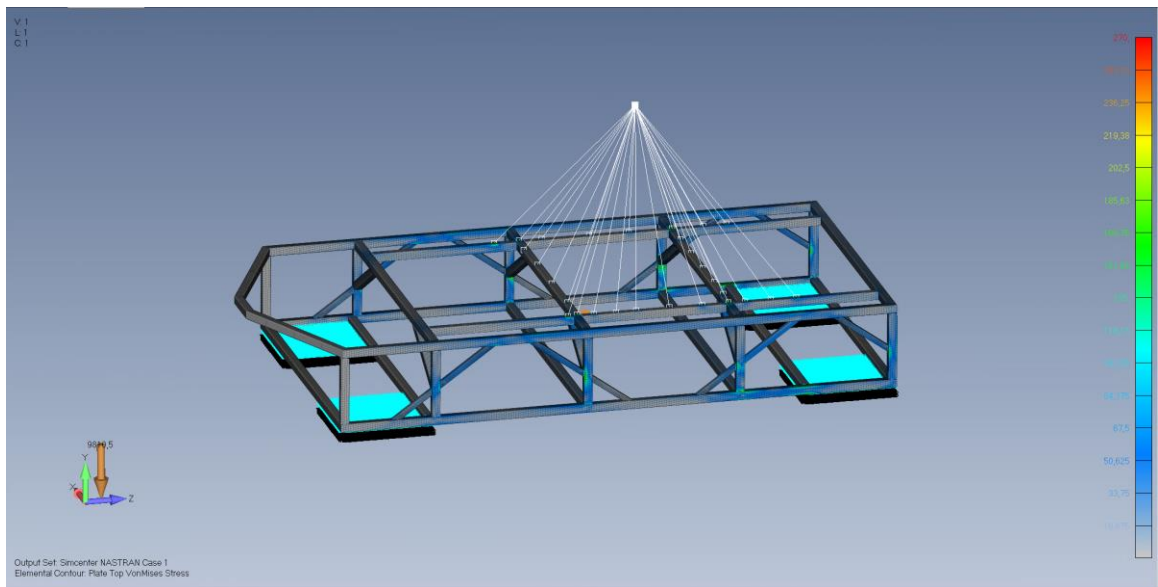
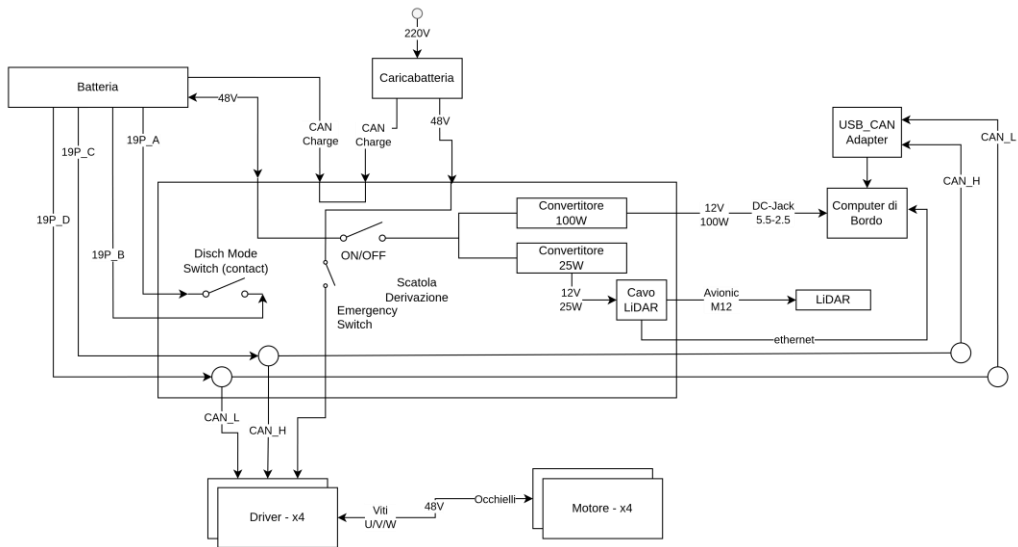


Fig. 6b: Simulazione FEM del telaio

Design del Prototipo – Progettazione Elettrica



4. Selezione Componenti

Selezione Componenti – Motoruote

- Compatte
- Facilmente installabili sul telaio
- Un solo fornitore per attuatori e gomme
- Encoder SIN/COS su ogni motore
- Termoresistenze su ogni motore
- Fine corsa per limiti di sterzo
- Flessibilità di posizionamento attuatori per sperimentazioni future



Fig. 8a: Logo azienda fornitrice

Fig. 8b: Modello 3D del prodotto

Selezione Componenti – Motoruote – Dimensionamento

Traction motor data		
Type : BR Brushless		
Power (S2 - 60 min)	W	500
Battery voltage	V	48
Speed	Rpm	2300
Frequency	Hz	76
Torque	Nm	2
Enclosure	IP	65
PMAC motor type	BD 090 - 50 - 4	

Traction gear data		
Gear ratio	i	1:21
Max accel. and braking torque	Nm	180
Nominal wheel torque	Nm	37
Max load on poliurethan wheel	Kg	550
Max load on solid rubber wheel	Kg	
Brake torque	Nm	5
Lubrication	Grease	

Steering gearmotor data		
Type : BR Brushless		
Power (S2 - 60 min)	W	300
Battery voltage	V	48
Motor Speed	Rpm	2800
Frequency	Hz	93
Motor torque	Nm	1
Gear ratio	i	1:45.56
Gear output torque	Nm	34
Enclosure	IP	65
Lubrication	Grease	
PMAC motor type	BD 090 - 25 - 4	



Fig. 8a: Logo azienda fornitrice
Fig. 8b: Modello 3D del prodotto

Selezione Componenti – LiDAR

- Compatto: 65x48x60mm
- Costo medio-basso
- Field of View: 360° x 59 °
 - Dotato di IMU
 - SDK open source, attivamente sviluppato e integrato in algoritmi di SLAM
- Cono cieco
 - Posizionato verso il basso

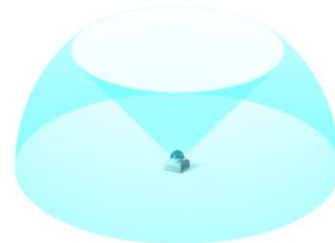


Fig. 10a: Dji LIVOX MID 360

Fig. 10b: Rappresentazione del campo visivo del sensore

Selezione Componenti – Alimentazione

- Batteria LiFePO4
 - Agli ioni di litio
 - Più sicuro delle normali batterie industriali
- Con Battery Management System comunicante in CANOpen
- Prodotta da una della aziende leader in Italia del settore

Nominal voltage	51.2	V
Nominal capacity	105	Ah
Chemistry type	LFP - Lithium iron phosphate (LiFePO4)	-
Maximum voltage	59.2	V
Minimum voltage	44	V
Discharge current	Constant: 105	A
Single Battery or system	Peak (t 60sec.): 210	A
Max. charging current	50	A
Depth of Discharge (DOD)	80	%
Max. regen voltage	56	V
Max regen Current	210	A
Total energy	5.38	kWh
Available energy (@80% DOD)	4.30	kWh
Operative temperature	-10°C / +45°C	
Self-discharge	3% / month @25°C	
	18% / month @50°C	



Fig. 12: Batteria



Selezione Componenti – Driver MicroPHASE Mini-Traction PWR

- Alimentazione a 48V in linea con le specifiche
- Configurabili per accettare encoder Sin/Cos come segnali di ingresso
- Controllati ad alto livello tramite protocollo CANOpen
- Controllo delle fasi dei motori tramite PID
- Tuning effettuato dalla ditta in fase di acquisto



Fig. 11: Driver Microphase

Selezione Componenti – Computer di Bordo

- LattePanda Sigma:
 - Dimensioni contenute
 - CPU x86, Intel Core i5-1340P: 12 core, 16 thread fino a 4.6Ghz
 - 16 GB RAM
 - Development board con GPIO
 - Alimentazione contenuta tra i 12 e i 20V. Consumi medi attorno i 90W
 - Compatibile con Windows e Linux
 - Microcontrollore Arduino Mega accoppiato



Fig. 13: Single board computer LattePanda Sigma

Adattatore USB-CANBus

- Soluzione commerciale a basso costo
- Rapidamente reperibile
- Traduce messaggi seriali in segnali analogici sul bus di campo
- Non implementa protocollo CANOpen
- ***Non implementa il driver per il protocollo SocketCAN del kernel Linux***



Fig. 13a: Adattatore USB-CAN-A

Fig. 13b: Schema di connessione PC – Dispositivo CAN

Vista Complessiva:

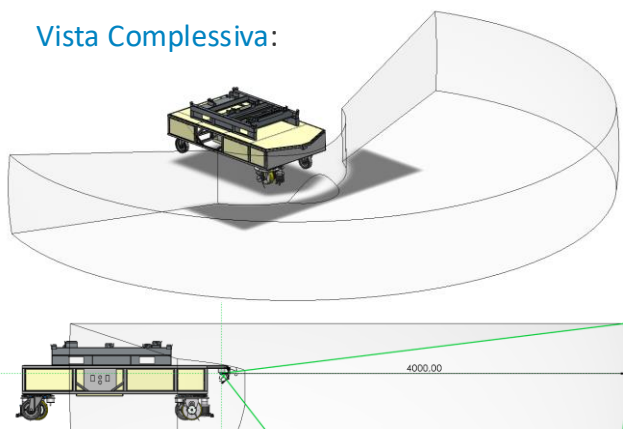


Fig. 14a: Telaio assemblato con motoruote e LiDAR montato. Simulazione del campo visivo

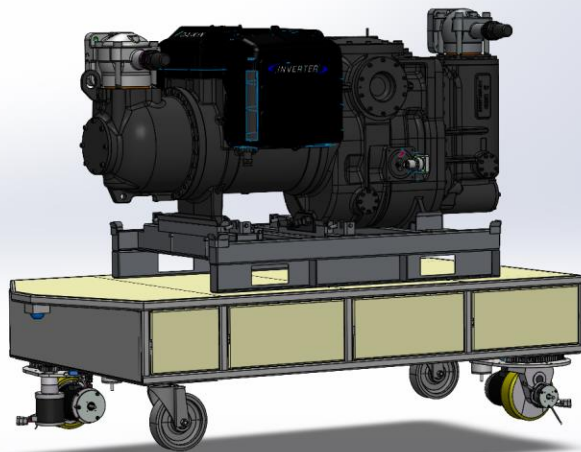


Fig. 14b: Telaio assemblato con motoruote e LiDAR montato. Al di sopra, compressore con la sua pedana di stoccaggio

Architettura di

5. Comunicazione e Controllo

Architettura di Comunicazione – Layer di Comunicazione

- Diversi device, con diverse interfacce:
 - Driver: CANOpen, protocollo CIA 301 e CIA 402
 - LiDAR: SDK, ROS2 wrapper. Connesso in sottorete con il PC
 - USB-CAN-A adapter: Char Device, con protocollo proprietario ma integrabile
 - BMS Batteria: CANOpen con implementazione parziale (solo errori e stato carica)
- Setup software sul SBC:
 - Linux (Ubuntu 24.04)
 - ROS2 come gestore dei task e dello strato applicativo
 - Docker per lo sviluppo e il deployment in ambiente isolato
 - Possibilità di integrazione di web-server per operazioni remote data la potenza della board

(un «file» sotto file-system Linux, disponibile sotto /dev/ttyUSB*)

Architettura di Comunicazione – CAN vs CANOpen

- CAN (Controller Area Network): livello fisico e data-link del modello ISO/OSI, sviluppato da Bosch per il settore automotive.
- Garantisce alta affidabilità e velocità di comunicazione
 - Fino a 1Mbit/s con protocollo classico, più elevate per le versioni avanzate (CAN FD)
 - Multi-Master con auto arbitraggio per evitare collisioni basate su ID dei pacchetti
 - Non contiene specifiche applicative
 - Auto rilevamento e gestione degli errori con CRC e ACK
- Il protocollo CANOpen descrive lo strato applicativo sopra il CAN.
- Elementi fondamentali:
 - Object Dictionary (OD)
 - Process Data Object (PDO)
 - Service Data Object (SDO)
 - Network Management (NMT)
 - Heartbeat
- Standardizza i profili di dispositivo:
 - Con protocolli aggiuntivi (CiA 402 per i driver elettrici)
 - Definendo gli Electrical Datasheets (EDS) che descrivono la mappatura degli indirizzi di memoria delle entry dell'OD

- Object Dictionary (OD): ad albero (id e sub-id) contenente i parametri del dispositivo, le configurazioni, i comandi e gli errori
- Process Data Object (PDO): pacchetti per messaggi senza overhead per trasmissioni rapide (es. letture sensori, assorbimento corrente) per controllo real-time
- Service Data Object (SDO): pacchetti per leggere/scrivere nelle aree di memoria dell'OD, per configurazione, diagnostica e tuning.
- Network Management (NMT): monitora lo stato dei nodi
- Heartbeat: per la verifica della presenza di un nodo nella rete

Architettura di Comunicazione – ROS2

- Framework software open-source per lo sviluppo di sistemi robotici
- Funzionamento distribuito tramite architettura a nodi e protocollo pub/sub
 - Gestione di architetture più complesse come servizi request/response e action server
- Gestione completa della comunicazione



Fig. 15: Logo ROS2

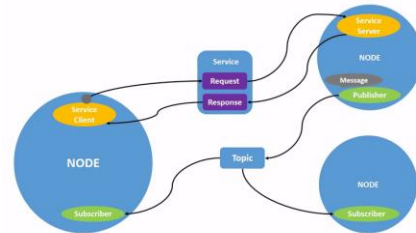


Fig. 16: Animazione della comunicazione di base tra nodi tramite topic mediante protocollo pub-sub

Architettura di Comunicazione – Vista ad alto livello

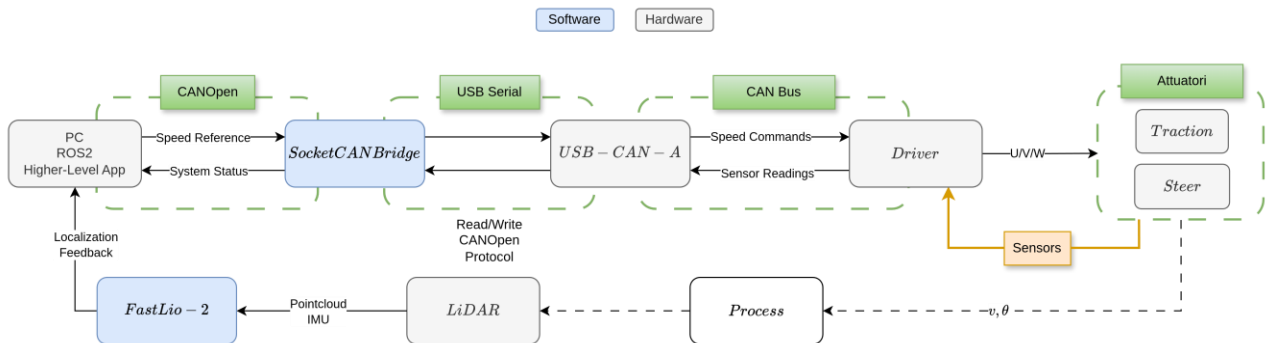


Fig. 17: Schema ad alto livello dell'architettura del sistema

- Il computer di bordo apre una CAN virtual network e si connette ad essa tramite il SocketCAN Bridge
- Il bridge apre e gestisce in modo concorrente la porta seriale dove è collegato l'adattatore USB
- Il computer, conoscendo la mappatura del driver, genera messaggi usando protocollo CANOpen (CIA 301, CIA402) che vengono «semplicemente» tradotti in un flusso di byte e inviati al driver tramite canale fisico
- Il LiDAR produce misurazioni pointcloud e le invia al computer assieme alle rilevazioni inerziali dell'IMU interna
- Driver e motori sono collegati tramite cavi di potenza e cavi per la lettura dei sensori

- Il Driver riceve segnali di riferimento in protocollo CANOpen e li traduce in segnali di corrente per controllare le fasi dei motori elettrici
- Le rilevazioni di velocità e di orientamento vengono usate dalla rappresentazione del modello per generare ulteriori riferimenti di velocità e orientamento

Architettura di Controllo - FastLIO-2

- Algoritmo di LiDAR-Inertial Odometry per posa 3D, agnostico rispetto al sensore utilizzato
 - No feature extraction
 - Output fino a 100Hz
 - Implementazione in ROS2 disponibile
 - Adatto ad applicazione indoor e outdoor
- Con filtro di Kalman interno per correggere distorsione di movimento
- Ottimizzazione della memoria con struttura dati ad albero bilanciata
- Sviluppato dal laboratorio HKU-MARS dell'università di Hong-Kong



Fig. 20a: Logo del laboratorio

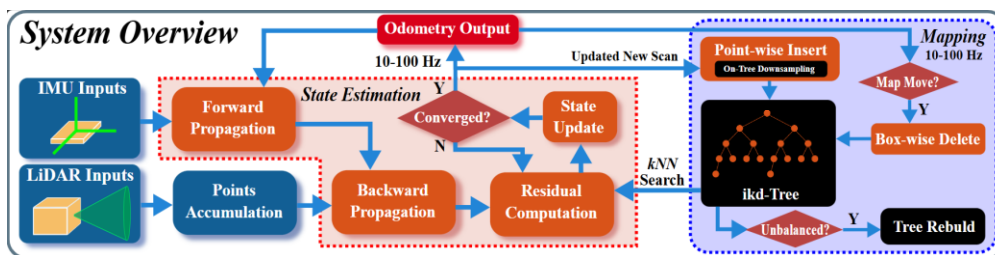


Fig. 20b: Pipeline di SLAM



Uso diretto delle feature del lidar: no identificazione geometrie

Durante la fase di **Forward Propagation** vengono integrate le misure dell'IMU per fare predizione sulla posa e la velocità del SDR del sensore, da un'istante t_k all'istante t_{k+1} . Viene letta accelerazione e velocità angolare tra i 200 e i 1000Hz ed eseguito un passo di integrazione usando un adeguato modello cinematico continuo, poi viene aggiornata la matrice della covarianza usando un modello dinamico lineare.

Durante la fase di **Backward Propagation** si associano le predizioni grezze della posa a ogni punto rilevato in quel k-esimo istante e viene fatta un'operazione di motion-compensation facendo delle trasformazioni all'indietro per sincronizzare tutti i punti prodotti dall'ultima scansione prodotta dal LiDAR

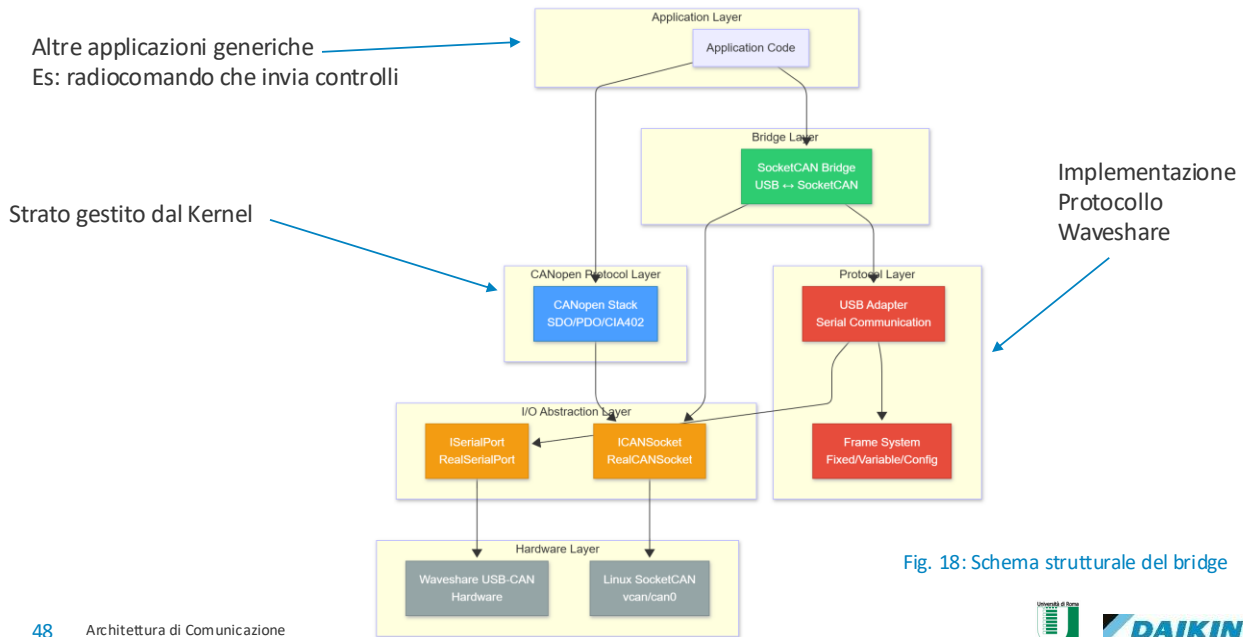
I punti vengono quindi registrati nell'albero, sul quale viene fatto kNN search per determinare per ogni punto quali sono i suoi vicini e identificare delle superfici planari. Se necessario, viene aggiornato l'albero per bilanciarlo.

Dopo la ricerca, vengono prodotte le informazioni necessarie al filtro di kalmann per stimare lo stato

Architettura di Comunicazione – SocketCAN e Waveshare Protocol

- SocketCAN: modulo del kernel Linux che implementa
 - Un driver per un controller CAN
 - Un'interfaccia di comunicazione basata su socket
 - Strumenti user-space per e API comuni a ogni linguaggio di programmazione moderno
- Supporta protocolli più alti come CANOpen
 - le cui librerie userspace usano gestiscono il canale come una socket
- L'adattatore utilizza un protocollo proprietario basato su frame
- ConfigFrame: per setup dei parametri seriali, del CANBus e del formato dati.
- DataFrame: per scambio dati, possono essere:
 - Fixed: taglia di 20 byte, richiede calcolo checksum per l'integrità
 - Variable: taglia variabile tra 5 e i 15 byte per scambio dati più veloce, accorpa più informazioni in un byte e ha un carattere di inizio e fine.
- Richiede configurazione diretta della porta seriale mediante librerie di sistema operativo

Architettura di Comunicazione – SocketCAN Bridge



48 Architettura di Comunicazione



- Interamente sviluppato da zero a partire dal protocollo Waveshare, riscrivendo in C++ delle librerie per la gestione dei singoli frame
- Gestisce una singola porta seriale in modo concorrente con un solo scrittore e molteplici lettori
- Apre una socket alla rete CAN virtuale (VCAN) ed effettua un forwarding doppio:
 - USB -> VCAN
 - VCAN -> USB
- Permette di rendere trasparente a qualsiasi programma user-space l'adattatore seriale e far diventare il PC a cui è connesso un nodo della rete CAN
- Implementato con un'architettura che fa astrazione sia della porta fisica che della socket per opportuno unit-testing delle singole funzionalità
- Sfrutta un sistema di callback per un'implementazione agnostica della gestione dei messaggi, lasciando all'utilizzatore la scelta di cosa fare quando si vuole inviare un messaggio e quando si vuole gestire un messaggio in arrivo
- Wrappata in un nodo ROS2 che ne gestisce il lifecycle e attivando una serie di servizi e topic per il monitoraggio della rete CAN e per inviare e leggere messaggi
- I comandi di controllo ai driver vengono inviati mediante opportuni nodi di

- interfaccia che forwardano ulteriormente i messaggi inviati a /cmd_vel (topic standard per l'invio di riferimento di velocità), trasformandoli in messaggi CAN
- Una serie di nodi viene lanciata in automatico all'avvio del sistema, che interrogano la rete per recuperare stato e letture del singolo driver e pubblicarle secondo le convenzioni scelte da ROS2 (es: catena cinematica con /tf)

Architettura di Controllo – Trajectory Tracking con PI + FeedForward

Obiettivo: Risolvere un problema di inseguimento di traiettorie

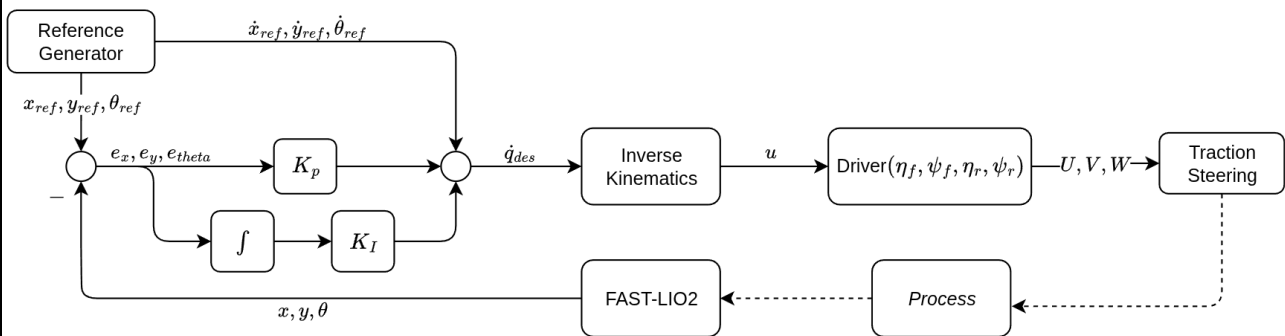


Fig. 19: Diagramma a blocchi Architettura di Controllo

Architettura di Controllo – Trajectory Tracking con PI + FeedForward

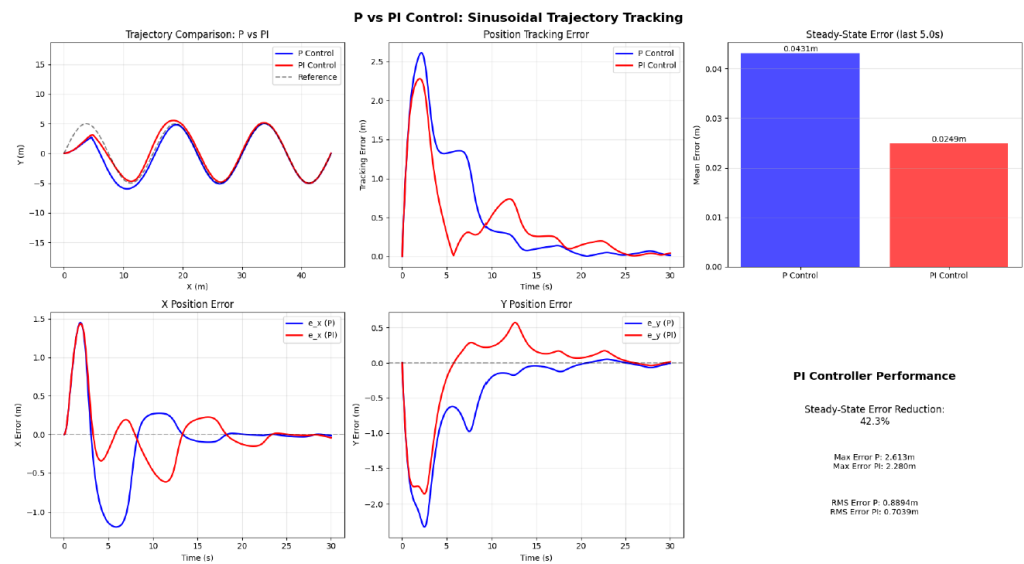


Fig. 19: Inseguimento Traiettorie sinusoidale

6. Stato dei Lavori

Cosa è stato realizzato e target a breve termine

Cosa è stato realizzato fino ad ora

- Progettazione meccanica in 3D con simulazioni FEM
- Progettazione schemi elettrici per il cablaggio
- Verificato il funzionamento dell'algoritmo FastLIO2 con LiDAR MID 360
- Implementazione di architettura di controllo su SBC

Cosa è stato realizzato fino ad ora

- Libreria `waveshare_cpp` per implementare il protocollo Waveshare in C++
- Libreria `SocketCANBridge` per fornire API per l'apertura e la gestione modulare del forwarding tra la CAN network del virtuale e il mezzo fisico
- Libreria che mappa la configurazione del file EDS (fornito dall'azienda) del driver per caricamento a run-time delle informazioni necessarie all'invio di messaggi

Cosa è stato realizzato fino ad ora

- Testata la comunicazione su CANBus tramite la libreria sviluppata, per il retrieving dello stato dei driver sia con messaggi CAN semplici che tramite SDO
- Testato l'invio di comandi ai driver per:
 - Cambiare configurazione dello stato del driver da inattivo ad operativo
 - Invio di set-point di posizione per angolo di sterzo, in open-loop
 - Invio di set-point di velocità ai motori, in open-loop

Cosa è in lavorazione

- Debugging del meccanismo di controllo e raccolta del feedback mediante PDO
- Monitoraggio stato batteria in tempo reale
- **Risoluzione criticità del modello**
- **Implementazione di un meccanismo di controllo del veicolo mediante l'invio di comandi da joystick e/o tastiera**

Sviluppi Futuri

- Integrare effettivamente meccanismi per la navigazione autonoma
 - SLAM con FastLIO2
 - EKF che fonde encoder e odometria da SLAM
 - Path-planning
 - Map-relocalization
 - Obstacle-Avoidance
- Integrazione di sistemi di Visual-SLAM accoppiando telecamere al LiDAR e/o l'utilizzo di più LiDAR per coprire maggiore campo visivo
- Meccanismi di Safety e policy di recovery
- Controllo remoto tramite Web App
- Ottimizzazione del design
- Sviluppo di un sistema accoppiato di rover per il trasporto della sezione ventilante e/o dell'intera macchina con aggancio frontale e posteriore o sistema di Kart/Trail
- Integrazione nei processi aziendali per il trasporto materiali dai depositi di magazzino alla linea mediante sistemi MES (Manufacturing Execution System) e di schedulazione della produzione

Grazie per l'attenzione

